

UNIVERSIDAD DEL VALLE DE GUATEMALA

Colegio Universitario



Laboratorio No. 2

Catedrático: Kevin Velazquez

Alejandro Anton – 221041

Juan Menéndez - 17444

Redes

Sección 10

Guatemala, 2026

Descripción de la práctica

El presente laboratorio tiene como objetivo principal comprender el funcionamiento de un modelo de capas y sus servicios mediante el desarrollo de programas que simulan cómo las redes detectan y corrigen errores al enviar datos. Debido a que en toda comunicación ocurren ruidos y errores de transmisión, se asume teóricamente que el medio utilizado no es confiable, lo que exige la implementación de mecanismos de control en la capa de Enlace.

Para llevar a cabo esta simulación, se diseñó una arquitectura de comunicación de extremo a extremo compuesta por un programa emisor y un programa receptor. Cumpliendo con los requisitos fundamentales, ambos extremos fueron desarrollados en distintos lenguajes de programación: Golang para el emisor y Python para el receptor. El sistema atraviesa lógicamente los servicios de las capas de Aplicación, Presentación, Enlace, Ruido y Transmisión para codificar cadenas de texto en ASCII binario y verificar su integridad.

Resultados

```
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/emisor
~/Documents/UVG/Redes/Lab2-Redes/emisor

jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/emisor$ go run main.go
--- INICIANDO EMISOR ---
Ingrese el mensaje a enviar: Hi
Seleccione el algoritmo (1: Detección [Fletcher], 2: Corrección [Hamming]):
1
[Presentación] Mensaje binario original: 0100100001101001
[Enlace] Aplicando Fletcher Checksum...
[Enlace] Trama con redundancia: 01001000011010011011000111111001

Ingrese la tasa de error (ej. 0.01 para 1/100, 0 para sin error): 0

=====
--- TRAMA FINAL PARA EL RECEPTOR ---
01001000011010011011000111111001
=====
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/emisor$
```

```
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/receptor
~/Documents/UVG/Redes/Lab2-Redes/receptor

jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/emisor jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/receptor$ python3 receptor.py
=== RECEPTOR (Python) - Lab 2 Redes ===
Algoritmo usado por el emisor (1: Fletcher, 2: Hamming): 1
Pega la trama binaria recibida del emisor: 01001000011010011011000111111001
-----
Trama recibida SIN ERRORES.
Mensaje: Hi
-----
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/receptor$
```

```
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/emisor$ go run main.go
--- INICIANDO EMISOR ---
Ingrese el mensaje a enviar: Hola
Seleccione el algoritmo (1: Detección [Fletcher], 2: Corrección [Hamming]):
1
[Presentación] Mensaje binario original: 01001000011011110110110001100001
[Enlace] Aplicando Fletcher Checksum...
[Enlace] Trama con redundancia: 010010000110111101101100011000011000010110101001

Ingrese la tasa de error (ej. 0.01 para 1/100, 0 para sin error): 0

=====
--- TRAMA FINAL PARA EL RECEPTOR ---
010010000110111101101100011000011000010110101001
=====
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/emisor$
```

```
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/receptor$ python3 receptor.py
=== RECEPTOR (Python) - Lab 2 Redes ===
Algoritmo usado por el emisor (1: Fletcher, 2: Hamming): 1
Pega la trama binaria recibida del emisor: 010010000110111101101100011000011000010110101001
-----
Trama recibida SIN ERRORES.
Mensaje: Hola
-----
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/receptor$
```

```
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/emisor$ go run main.go
--- INICIANDO EMISOR ---
Ingrese el mensaje a enviar: UVG Redes 2026
Seleccione el algoritmo (1: Detección [Fletcher], 2: Corrección [Hamming]):
1
[Presentación] Mensaje binario original: 01010101010110010001110010000001010010011001010110010001100101011100110
010000000110010001100000011001000110110
[Enlace] Aplicando Fletcher Checksum...
[Enlace] Trama con redundancia: 0101010101011001000111001000000101001001100101011001000110010101110011001000000
110010001100000011001000110110111100101011111
Ingrese la tasa de error (ej. 0.01 para 1/100, 0 para sin error): 0
=====
--- TRAMA FINAL PARA EL RECEPTOR ---
0101010101011001000111001000000101001001100101011001000110010101110011001000000011001000110000001100100011011011
1100101011111
=====
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/emisor$
```

```
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/receptor$ python3 receptor.py
=== RECEPTOR (Python) - Lab 2 Redes ===
Algoritmo usado por el emisor (1: Fletcher, 2: Hamming): 1
Pega la trama binaria recibida del emisor: 010101010101100100011100100000010100100110010101100100011001010111001
10010000000110010001100000011001000110110111100101011111
-----
Trama recibida SIN ERRORES.
Mensaje: UVG Redes 2026
-----
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/receptor$
```

```
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/emisor$ go run main.go
```

```
--- INICIANDO EMISOR ---
```

```
Ingrese el mensaje a enviar: Hola
```

```
Seleccione el algoritmo (1: Detección [Fletcher], 2: Corrección [Hamming]):
```

```
2
```

```
[Presentación] Mensaje binario original: 01001000011011110110110001100001
```

```
[Enlace] Aplicando Código de Hamming...
```

```
[Enlace] Trama con redundancia: 1100100110000111011101101100010100001
```

```
Ingrese la tasa de error (ej. 0.01 para 1/100, 0 para sin error): 0.10
```

```
=====
```

```
--- TRAMA FINAL PARA EL RECEPTOR ---
```

```
11011001100000010111101101100010000001
```

```
=====
```

```
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/emisor$
```

```
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/receptor$ python3 receptor.py
```

```
=== RECEPTOR (Python) - Lab 2 Redes ===
```

```
Algoritmo usado por el emisor (1: Fletcher, 2: Hamming): 2
```

```
Pega la trama binaria recibida del emisor: 11011001100000010111101101100010000001
```

```
-----
```

```
Se detectó un error en la posición 36 y fue CORREGIDO.
```

```
Trama original limpia: 01001000000011110110110001000101
```

```
Mensaje: 01001000000011110110110001000101
```

```
-----
```

```
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/receptor$
```

```
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/emisor$ go run main.go
```

```
--- INICIANDO EMISOR ---
```

```
Ingrese el mensaje a enviar: Hola, esto es un secreto
```

```
Seleccione el algoritmo (1: Detección [Fletcher], 2: Corrección [Hamming]):
```

```
2
```

```
[Presentación] Mensaje binario original: 0100100001101111011011000110000100101100001000000110010101110011011101000
```

```
110111100100000011001010111001100100000011101010110111000100000011100110110010101100011011100100110010101110100011
```

```
01111
```

```
[Enlace] Aplicando Código de Hamming...
```

```
[Enlace] Trama con redundancia: 0001100110000111011110110110001010000100101100001000000110010101111001101110100011
```

```
011110010000001100101011100110010000001110101101110001000000111001101100101011000110111001001100101011101000110
```

```
1111
```

```
Ingrese la tasa de error (ej. 0.01 para 1/100, 0 para sin error): 0
```

```
=====
```

```
--- TRAMA FINAL PARA EL RECEPTOR ---
```

```
000110011000011101111011011000101000010010110000100000011001010111100110111010001101111001000000110010101110011001
```

```
000000111010110110111000100000011100110110010110001101110010011001010111010001101111
```

```
=====
```

```
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/emisor$
```

```
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/receptor$ python3 receptor.py
```

```
=== RECEPTOR (Python) - Lab 2 Redes ===
```

```
Algoritmo usado por el emisor (1: Fletcher, 2: Hamming): 2
```

```
Pega la trama binaria recibida del emisor: 01010101010000000110011000011101111011011000101000010010110000100000011
```

```
001010111100110111010001101111001000000110010101100110010000001100110110010101100011011
```

```
1001001100101011101000110111
```

```
-----
```

```
Se detectó un error en la posición 50 y fue CORREGIDO.
```

```
Trama original limpia: 0010010000001100110000111011101101100010100101011000010000011001010111100110111010001101
```

```
111001000000110010101110011000000001110101101110001000000111001101100101011000110111001001100101011101000110111
```

```
1
```

```
Mensaje: 001001000000110011000011101110110110001010010101100001000001100101011110011011101000110111100100000011
```

```
0010101110001000000011101011011011100010000001110011011001010110001101110010011001010111010001101111
```

```
-----
```

```
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/receptor$ python3 receptor.py
```

```
=== RECEPTOR (Python) - Lab 2 Redes ===
```

```
Algoritmo usado por el emisor (1: Fletcher, 2: Hamming): 1
```

```
Pega la trama binaria recibida del emisor: 0010010000001100110000111011101101100010100101010010110000100000110010101
```

```
11100110111010001101111001000000110010101100110000000011101011011100010000001110011011001010110001101110010011
```

```
001010111010001101111
```

```
-----
```

```
ERROR: se detectó un error en la trama. Trama descartada.
```

```
jf-omen-fedora@fedora:~/Documents/UVG/Redes/Lab2-Redes/receptor$
```

Discusión

¿ES POSIBLE "ENGAÑAR" AL ALGORITMO?

- Engañando a Fletcher Checksum: Este algoritmo divide la trama en bloques y realiza sumas acumulativas. Si se modifican varios bits de manera deliberada para que los errores se "cancelen" entre sí dentro de la suma matemática (una colisión), el receptor calculará exactamente el mismo checksum que el original. En este escenario, el programa receptor asume erróneamente que la trama está intacta y la procesa como "SIN ERRORES", entregando un mensaje corrupto.
- Engañando al Código de Hamming: La debilidad estructural de Hamming radica en su diseño enfocado a corregir un único bit erróneo. Al introducir dos o más errores en la trama, el recálculo de los bits de paridad arroja un resultado distinto de cero, pero apunta a una posición equivocada. El algoritmo, creyendo que hace lo correcto, invierte un bit que originalmente estaba sano. Esto no solo falla en detectar el problema real, sino que daña aún más el mensaje sin alertar al usuario de que la corrección fue fallida.

ANÁLISIS COMPARATIVO

Característica	Fletcher Checksum	Código de Hamming
Dificultad de Implementación	Baja. Utiliza operaciones aritméticas sencillas	Moderada. Requiere manejo de posiciones en potencias de 2 e intercalado de bits mediante operaciones lógicas
Velocidad Operativa	Muy alta. Procesa bloques enteros de datos secuencialmente	Menor. Debe evaluar la trama iterativamente para armar y verificar los bits de paridad
Redundancia	Fija y baja. Exige exactamente 16 bits extra sin importar el tamaño del mensaje	Dinámica y alta. Exige bits extra generando mucho overhead en mensajes cortos

¿QUÉ ALGORITMO TUVO UN MEJOR FUNCIONAMIENTO?

El "mejor" funcionamiento depende del objetivo de la red. Para garantizar que la información recibida sea 100% fidedigna (evitando falsas correcciones), Fletcher tuvo un desempeño superior al descartar inmediatamente las tramas corruptas. Sin embargo, para mantener el flujo continuo de la aplicación sin interrupciones, Hamming funcionó mejor en los escenarios de ruido mínimo (un solo error).

Ninguno de los dos algoritmos básicos es ideal para canales con muchísimo ruido, pero Fletcher demostró ser más confiable ante tasas de error altas. Si la cantidad de ruido aumenta drásticamente, Fletcher simplemente detectará el desorden y descartará las tramas. Hamming no es flexible ante altas tasas de error, ya que colapsa si hay más de un bit alterado, entregando datos corruptos al usuario.

¿CUÁNDO ES MEJOR UTILIZAR UN ALGORITMO DE DETECCIÓN DE ERRORES EN LUGAR DE UNO DE CORRECCIÓN?

Se debe usar Detección (Fletcher): En redes bidireccionales donde el medio es relativamente estable y el costo de pedir que se reenvíe el mensaje es bajo (como en una conexión de red local o Wi-Fi estable). Es preferible descartar un paquete y pedirlo de nuevo que arriesgarse a usar información corrupta.

Se debe usar Corrección (Hamming): En transmisiones unidireccionales o donde pedir un reenvío es imposible o demasiado lento. Ejemplos clásicos son las comunicaciones con satélites, transmisiones de video en vivo (donde no hay tiempo de reenviar cuadros perdidos), o al leer datos directamente de un disco duro.

CONCLUSIONES

- Se logró implementar exitosamente una arquitectura de comunicación por capas usando dos lenguajes distintos, demostrando que los servicios de cada capa pueden mantenerse independientes del lenguaje de implementación mientras se respete el contrato de datos entre ellas.
- Fletcher checksum resultó ser el algoritmo más confiable para garantizar la integridad de los datos: al ser un mecanismo de detección puro, nunca entrega un mensaje corrupto sin advertirlo, sacrificando disponibilidad (descarta la trama) a cambio de certeza.
- El código de Hamming demostró ser eficiente únicamente en su caso de diseño (un solo bit erróneo), pero representa un riesgo real en canales con ruido moderado o alto: al intentar corregir dos o más errores, no solo falla en identificarlos correctamente, sino que puede alterar un bit sano y entregar un mensaje corrupto sin ninguna alerta, lo cual es más peligroso que no corregir nada.
- La elección entre detección y corrección de errores no depende del algoritmo en sí, sino del contexto de la red: en canales bidireccionales con reenvío barato (Wi-Fi, LAN), la detección con descarte y retransmisión es la opción más segura; en canales unidireccionales o de alta latencia (satélites, streaming en vivo), la

corrección es indispensable pese a sus limitaciones, ya que no reenviar no es una opción.

- Existe un compromiso directo entre redundancia y capacidad de recuperación: Fletcher mantiene un overhead fijo de 16 bits independientemente del tamaño del mensaje, mientras que Hamming necesita más bits de paridad a medida que crece el mensaje, pero a cambio ofrece la posibilidad de recuperar la información sin retransmisión — una decisión de diseño que en la práctica depende del ancho de banda disponible y la tolerancia a errores del sistema.